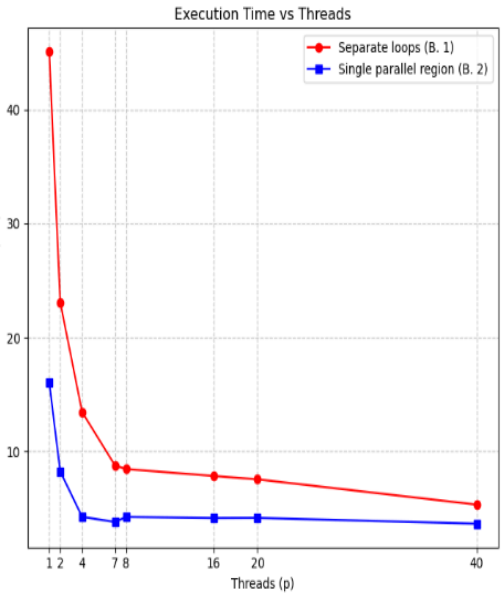
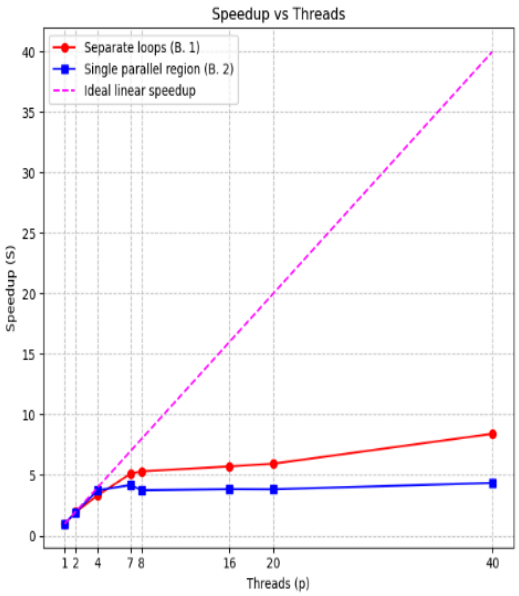
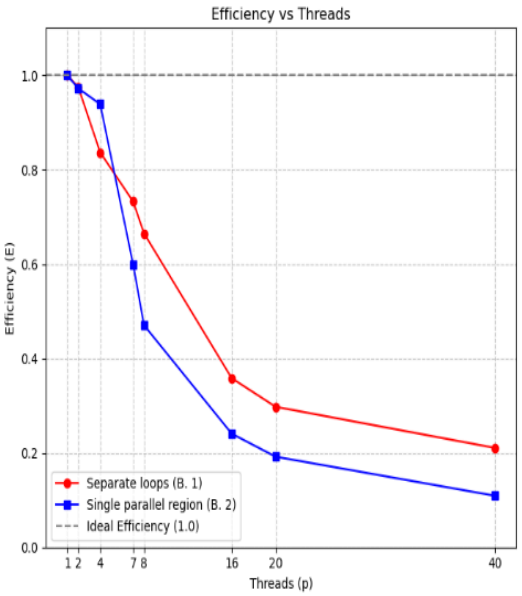
Замеры времени, расчёты ускорения и эффективности

N=2000, eps = 1e-5, max_iter=10000	Количество потоков														
	2			4		7		8		16		20		40	
	T_1	T_2	S_2	T_4	S_4	T_7	S_7	T_8	S_8	T_16	S_16	T_20	S_20	T_40	S_40
B. 1	46,459748	23,410075	1,984604	813,343925	3,4817153	8,916768	5,21037981	8,554687	5,4309114	7,318764	6,34803199	5,804217	8,00448156	4,661378	9,966955694
B. 2	16,557797	8,449535	1,9596104	5,89981	2,8064966	4,09801	4,04044816	4,279594	3,8690111	3,927503	4,21585852	3,727175	4,44245225	3,131907	5,286809921
	Эффективность														
	Ef. 1		Ef.2		Ef. 4		Ef. 7		Ef. 8		Ef. 16		Ef. 20		Ef. 40
B. 1	1		0,9923024		0,8704288		0,74433997		0,6788639		0,39675199		0,40022407		0,2491738924
B. 2	1		0,9798052		0,7016241		0,57720688		0,4836263		0,26349115		0,22212261		0,132170248

Графики зависимостей







Статистика для omp parallel for schedule()

schedule type, n_threads = 8	static	static, 1	static, 32	static, 128	dynamic	dynamic, 8	dynamic, 32	guided	guided, 16	auto
time	8.733840	8.047303	8.133916	8.547338	9.135570	9.937977	10.059155	9.644042	8.653465	8.019534

Вывод о целесообразности использования вариантов программы

После выполнения Лабораторной работы №2 «Параллельная реализация решения системы линейных алгебраических уравнений с помощью OpenMP» можно сделать вывод что вариант 2 (единая параллельная область *#pragma omp parallel* с ручным распределением нагрузки) является целесообразным.

Анализ данных
При переходе с 1 потока на 2 время первого варианта сокращается практически вдвое, а при переходе на 40 потоков коэффициент ускорения составляет 9,96. Для второго варианта программы коэффициент ускорения при 40 потоках составляет 5,28. Коэффициент ускорения первого варианта по своему значению выше чем для второго практически для всех потоков, но это не говорит нам об эффективности первого варианта. Время выполнения показывает что даже при использовании 1 потока вариант 1 менее эффективен чем вариант 2 (46,45с против 16,57), ту же тенденцию можно наблюдать и при увеличении количества потоков, например, при использовании 40 потоков время для вариантов: 4,66 против 3,13.

Обоснование
Разница в производительности и эффективности между двумя реализациями объясняется следующими архитектурными особенностями:

1. Снижение накладных расходов на создание потоков: В Варианте 1 параллельная область создается и завершается дважды внутри каждой итерации цикла while (отдельно для вычисления произведения матрицы на вектор и отдельно для вычисления нормы невязки). При большом количестве итераций (до 10 000) затраты времени на постоянную инициализацию и синхронизацию потоков становятся сопоставимы с полезной вычислительной нагрузкой. В Варианте 2 поток создается единожды на весь период работы алгоритма, что практически полностью нивелирует эти системные задержки.
2. Эффективность работы с кэш-памятью: Использование единого параллельного региона (Вариант 2) позволяет потокам более эффективно удерживать данные в L1/L2 кэшах процессора. Поскольку данные векторов не «сбрасываются» между двумя разными параллельными циклами (как это происходит в Варианте 1), уменьшается количество промахов кэша и снижается нагрузка на шину памяти.
3. Локальность данных и синхронизация: Ручное распределение нагрузки во втором варианте дает программисту контроль над тем, как данные распределяются между ядрами. В то время как первый вариант полагается на неявные барьеры синхронизации OpenMP в конце каждого цикла for, второй вариант использует лишь необходимые точечные барьеры внутри единой области, что позволяет ядрам работать более автономно.
4. Сравнение базовой производительности: Тот факт, что даже на 1 потоке Вариант 2 работает в 2.8 раза быстрее (16.06с против 45.04с), указывает на наличие в первой реализации избыточных операций или менее оптимального обращения к памяти, которые проявляются именно при частых перезапусках параллельных секций.

Итог: Несмотря на более высокий относительный показатель ускорения у первого варианта (за счет очень медленного старта на одном потоке), Вариант 2 является безусловно предпочтительным, так как обеспечивает минимальное абсолютное время решения задачи и более рациональное использование вычислительных ресурсов.